

PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Implementation and Comparison of LL(1), LR(0), and SLR(1) Parsers for Arithmetic Expressions

Student / Team Information

Team Name: Team #	Avengers
Team member 1 (Team Lead) (Last Name, name: student ID: email, picture):	Giri, Sneha – 2023793 sneha20giri03@gmail.com 
Team member 2 (Last Name, name: student ID: email, picture):	Uniyal, Unnati – 2024128 uniyalunnati@gmail.com 

Team member 3 (Last Name, name: student ID: email, picture):	Rais, Khulood Mohammad – 2023411 khuloodrais@gmail.com 
Team member 4 (Last Name, name: student ID: email, picture):	Tiwari, Saumyadeep– 2024104 saumyadeeptiwari@gmail.com 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

Parsing is one of the most important phases in compiler design because it determines whether a given input follows the rules of a language grammar. While studying compiler concepts, we learn about different parsing techniques such as LL(1), LR(0), and SLR(1). However, these techniques are often understood only theoretically, and their practical differences are not always clear.

Arithmetic expressions are a fundamental part of programming languages and provide a simple but meaningful way to understand syntax analysis. Different parsing methods handle grammars in different ways, and not all grammars suitable for LR parsing can be handled by LL(1). This project is motivated by the need to clearly understand how these parsing techniques differ in terms of grammar requirements, parsing table construction, complexity, and error detection.

By implementing LL(1), LR(0), and SLR(1) parsers for arithmetic expressions, this project will allow a direct comparison of top-down and bottom-up parsing strategies. Instead of only studying algorithms from textbooks, the goal is to construct parsing tables, implement stack-based parsing mechanisms, and observe how each method processes the same input.

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

In modern compilers, syntax analysis is usually handled by automated parser generators such as YACC. These tools internally construct LR-based parsing tables and generate efficient parsers automatically. As a result, developers rarely implement parsing algorithms manually.

Most educational resources explain LL(1), LR(0), and SLR(1) parsing techniques using small examples, but they are not often implemented and compared within a single system. Typically, students implement only one parsing strategy in isolation.

Arithmetic expression parsing is commonly demonstrated using operator precedence parsing or recursive descent parsing. However, direct comparison between LL(1), LR(0), and SLR(1) using the same grammar is less frequently explored in practical coursework.

This project differs from standard approaches by manually implementing all three parsing techniques and analysing their behaviour, grammar constraints, and table construction differences on the same set of arithmetic expressions.

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

The main goal of this project is to design and implement syntax analysers for arithmetic expressions using three different parsing techniques: LL(1), LR(0), and SLR(1). The project also aims to compare their behaviour, grammar handling capability, and parsing efficiency.

The first milestone will be defining a suitable grammar for arithmetic expressions that includes operators such as addition, subtraction, multiplication, division, parentheses, and identifiers. The grammar will be modified where necessary to make it compatible with LL(1) parsing by removing left recursion and applying left factoring.

The second milestone will involve constructing the LL(1) parser. This includes computing FIRST and FOLLOW sets and building the predictive parsing table. We will also focus on LR(0) parsing. LR(0) items will be constructed, canonical collection of LR(0) states will be generated, and the parsing table will be built.

The third milestone will involve constructing the SLR(1) parser using LR(0) items combined with FOLLOW sets to resolve conflicts where possible.

Finally, all three parsers will be tested using multiple arithmetic expressions. Their performance, grammar flexibility, and conflict handling will be compared and documented.

Project Approach (3 pts)

(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

The project will begin by selecting a standard arithmetic expression grammar. Since LL(1) parsing cannot handle left-recursive grammars, the grammar will first be transformed into an LL(1)-compatible form. FIRST and FOLLOW sets will be calculated, and a predictive parsing table will be constructed. A stack-based LL(1) parser will then be implemented.

For LR(0), the augmented grammar will be created first. LR(0) items will be generated, and the canonical collection of LR(0) states will be constructed using closure and goto functions. Based on these states, the LR(0) parsing table will be formed.

For SLR(1), the same LR(0) item sets will be used, but FOLLOW sets will be applied to reduce and shift-reduce conflicts where possible.

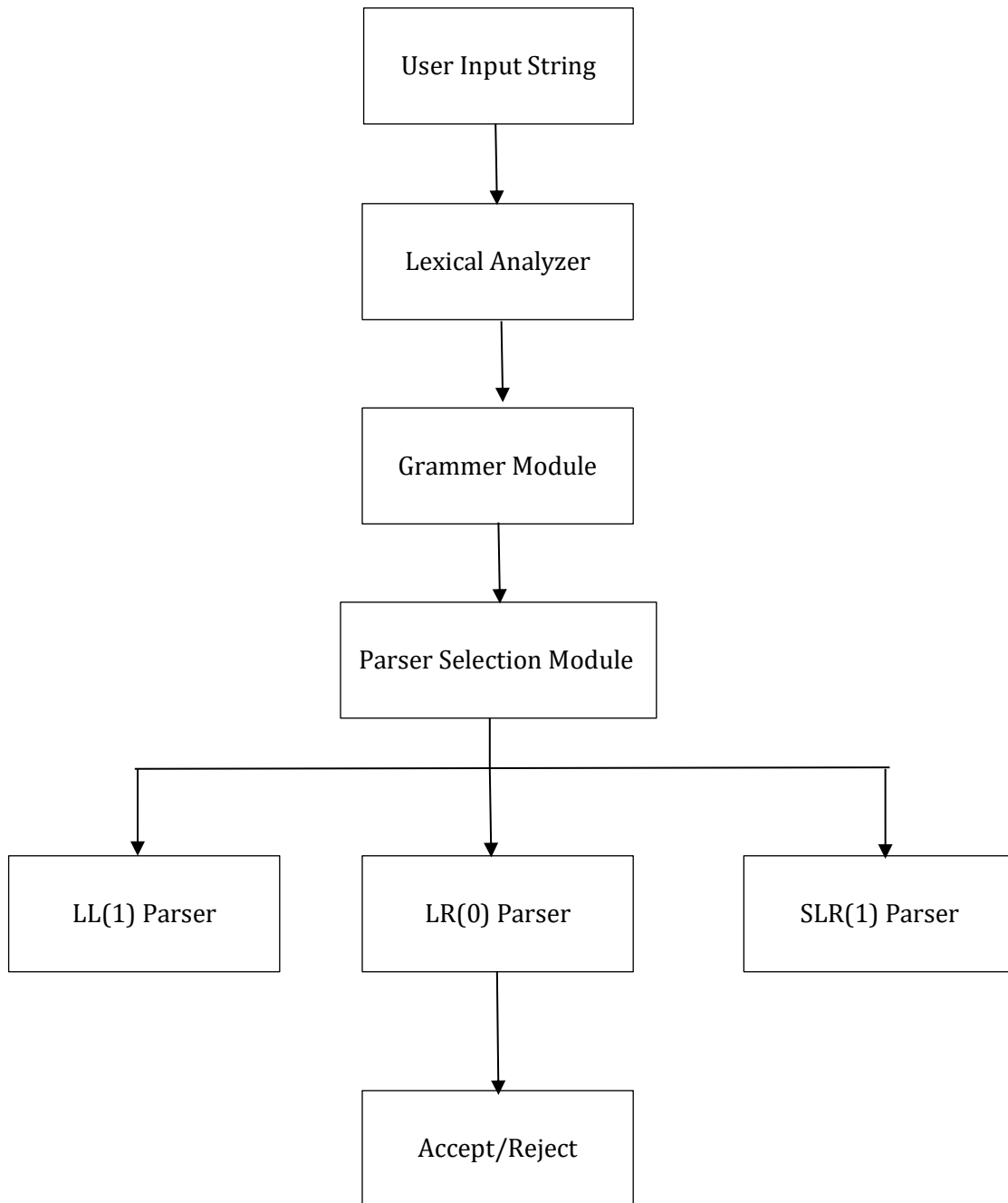
All parsers will follow a stack-based implementation. The input string will be processed symbol by symbol until it is either accepted or rejected.

The final stage will include testing and comparison, focusing on table size, conflicts, grammar limitations, and parsing steps required for the same input expression.

The implementation can be done in C++ or Java using basic data structures such as stacks, arrays, and sets.

System Architecture (High Level Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).



Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

The final outcome of this project will be three working syntax analysers for arithmetic expressions implemented using LL(1), LR(0), and SLR(1) parsing techniques. Each parser will be capable of validating whether a given arithmetic expression follows the defined grammar.

The project will deliver:

- A clearly defined grammar for arithmetic expressions
- FIRST and FOLLOW set computation for LL(1) and SLR(1)
- LL(1) parsing table
- Canonical collection of LR(0) items
- LR(0) parsing table
- SLR(1) parsing table
- Stack-based implementation of all three parsers
- Test cases demonstrating valid and invalid expressions
- Comparative analysis of conflicts, table sizes, and limitations

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words)

The project assumes that the input arithmetic expressions consist of identifiers, integers, operators (+, -, *, /), and parentheses. It is assumed that tokens are correctly separated during lexical analysis.

The grammar used will follow standard precedence rules for arithmetic expressions. The project focuses only on syntax validation and does not evaluate the expression semantically.

It is also assumed that the input expressions are of reasonable length and do not include advanced language features such as functions, unary operators, or floating-point precision handling.

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

1. Alfred V. Aho – Lecture Notes on Syntax Analysis.
2. GeeksforGeeks – LL(1) Parsing
<https://www.geeksforgeeks.org/compiler-design-ll1-parser-in-python/>
3. GeeksforGeeks – LR Parsing
<https://www.geeksforgeeks.org/lr-parser/>
4. Tutorialspoint – SLR Parsing
https://www.tutorialspoint.com/compiler_design/compiler_design_slr_parser.html